

Автоматизация деплоя, миграции, интеграции для self-hosted таблиц

Презентация по итогам итерации №4

Заказчик: Заславский М.М.

Исполнители: Попандопуло А.Г. гр. 1304

Марков М.М., Сычев Н.С., Гладилин Е.О., Журавлев Д.В., Яковлев Д.С. гр.

План текущей итерации

Нагрузочное тестирование: реализовать сценарий эмуляции типовой нагрузки с метриками отклика и потребления ресурсов.

Production-ready конфигурация: поддержка HTTPS-режима, тематическое разбиение env-параметров, ограничение ресурсов контейнеров.

Integration-тесты: pytest-сценарии на ключевые CLI-команды.

Финальные доработки: по обратной связи со стороны заказчика.

Результаты текущей итерации

Нагрузочное тестирование: сценарии Locust (stepped, smoke), команда `manage.py loadtest run`, отчёты HTML+CSV; референсный прогон — 38 000 запросов на 50→300 пользователей, 0 ошибок.

HTTPS: подготовлены два варианта — TLS на внешнем reverse проху и TLS внутри стека через `compose-overlay` с подменой `nginx`-конфига.

Env-аудит: `deploy/.env` разбит на тематические секции, добавлены `CRON_INTERVAL` и `SYNC_INTERVAL`, прописаны ограничения ресурсов на все контейнеры.

Integration-тесты: `pytest`-сценарии под CLI-команды (`users` / `upload` / `import` / `backup` / `deploy status`) в `table/tests/` для локального прогона.

Финальные доработки: в соответствии с указаниями, внесены правки по части документации и развертывания системы.

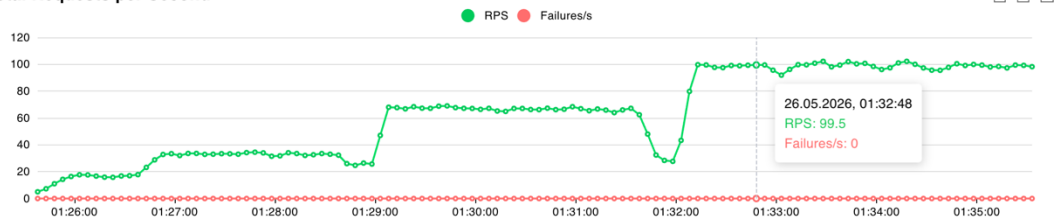
Итоги проекта

- стек Nextcloud + OnlyOffice развернут одной командой, эксплуатация (пользователи, таблицы, бэкапы, мониторинг, импорт, нагрузочное тестирование) — через единый CLI `manage.py / bin/table-cli`.
- Закрыты установленные сценарии использования, среди которых: сбор данных через формы, управление доступом, импорт CSV из внешних систем, нагрузочные замеры и резервное копирование.
- Поддерживается production-ready развертывание с HTTPS и настраиваемыми лимитами ресурсов
- Проведено нагрузочное тестирование и определены количественные тенденции потребления ресурсов системы

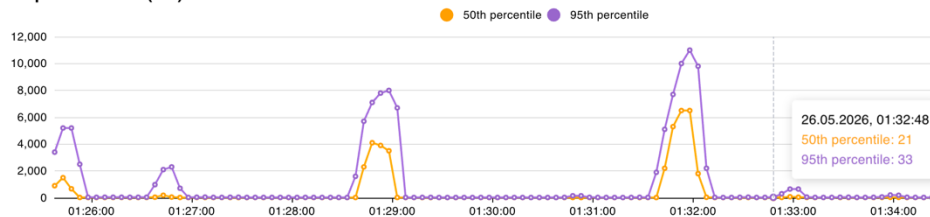
Нагрузочное тестирование — результаты Locust

Charts

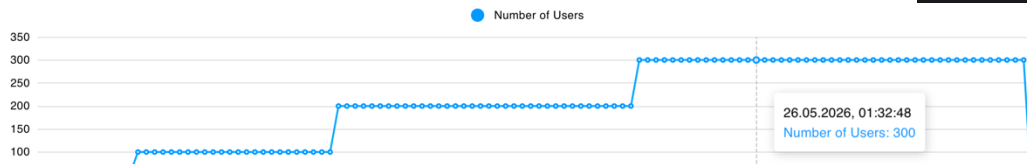
Total Requests per Second



Response Times (ms)



Number of Users



Loadtest results: table/loadtest/results/2026-05-25T21-24-02
Loadtest start (folder ts): 2026-05-25 21:24:02

Snapshots: 145

First snapshot: 2026-05-25 21:24:00.051628

Last snapshot: 2026-05-25 21:42:17.922075

Stage	samples	app CPU%	app RAM MB	cron CPU%	OO CPU%	total CPU%	total MB	API ms
baseline	1	0.0	553.7	8.9	0.3	9.2	2120.3	13.0
stage1 (50 users)	8	97.0	606.5	0.0	0.2	138.8	2183.1	70.1
stage2 (100 users)	16	89.1	610.6	3.6	0.9	125.3	2186.8	23.2
stage3 (200 users)	22	107.3	588.8	22.4	0.7	175.9	2196.3	170.6
stage4 (300 users)	30	159.1	628.9	8.0	0.6	225.0	2225.2	162.0
cooldown	68	1.4	542.3	1.2	0.6	3.6	2127.1	11.9

HTTPS

HTTPS

Поддерживаются два варианта терминов TLS.

Вариант А - TLS на внешнем reverse proxy

Стек поднимается как есть, HTTPS термируется внешним reverse proxy перед ним.

Параметры в `deploy/.env` :

- `NEXTCLOUD_HOSTNAME` - публичное DNS-имя, по которому ходит браузер
- `NEXTCLOUD_TRUSTED_PROXIES` - CIDR-подсеть(и) прокси (JSON-массив)

Локальный стенд для проверки с Caddy и self-signed сертификатом - в `playground/caddy-tls-test/` :

```
./bin/table-cli deploy up
docker compose -f playground/caddy-tls-test/docker-compose.yml up -d
curl --no-proxy '*' -k https://nextcloud.localhost:8443/status.php
```

Вариант Б - TLS внутри стека

`nginx-server` сам слушает 443 и термирует TLS.

```
# 1. Положить сертификат в deploy/certs/ (server.crt + server.key).
# Для локальной проверки можно сгенерировать self-signed dev-утилитой:
bash playground/https-test/generate-self-signed.sh nextcloud.localhost
```

```
# 2. Поднять стек с HTTPS-overlay
docker compose \
  -f deploy/docker-compose.yml \
  -f deploy/docker-compose.https.yml \
  up -d
```

```
# 3. Проверка
curl -k https://nextcloud.localhost:8443/status.php
```

HTTPS-порт меняется через `NEXTCLOUD_HTTPS_PORT` в `deploy/.env` (по умолчанию 8443). HTTP-порт 8080 остаётся открытым и отдаёт `301 → https://`.